

YANG **Message Keys** for Message Broker Integration

draft-ietf-nmop-yang-message-broker-message-key-02

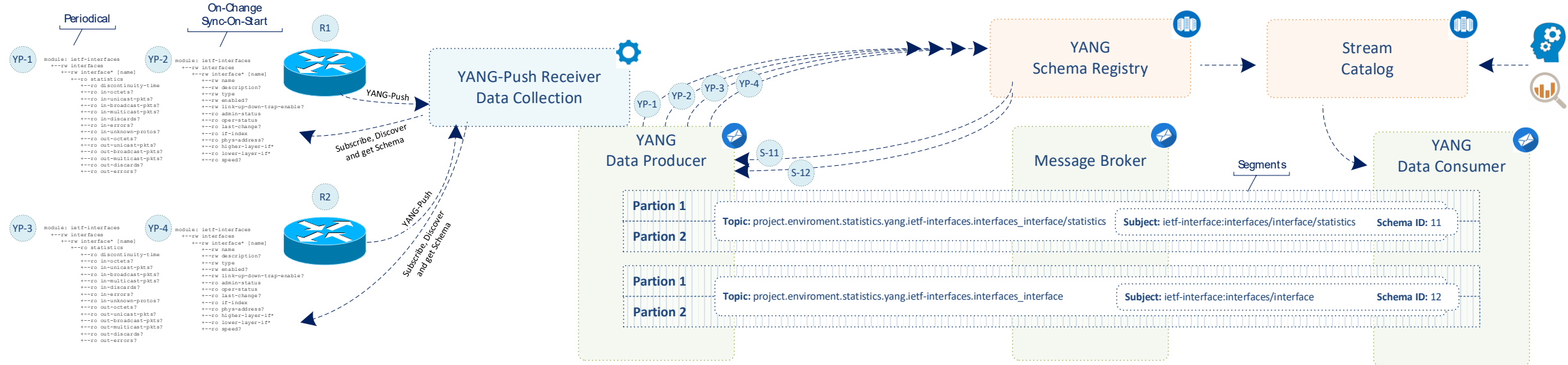
A mechanism to define a unique message key for a
YANG to message broker integration and a topic addressing scheme
based on YANG-Push subscription type and a YANG index

thomas.graf@swisscom.com
ahmed.elhassany@swisscom.com
alex.huang-feng@insa-lyon.fr
benoit@everything-ops.net
paolo@ntt.net

23. July 2026

We aim for a fully YANG aware Data Pipeline

From network nodes to message broker nodes



- At the YANG data producer, thanks to the YANG awareness to the YANG data producer and the YANG schema registry, **for each unique YANG schema tree a message broker topic and subject is being created, and messages are being indexed and hashed according to the message key.**
- At the message broker, thanks to the dedicated topic per YANG schema and its message key, **YANG state metrics can be compacted to the latest state; reducing the number of stored messages.**
- At the YANG data consumer, thanks to the YANG awareness of the stream catalog and consumer, a specific topic, partition and subject can be consumed for the interested YANG metrics; **reducing the number of messages to be consumed.**

Message Broker Integration

Document Relationships

These are the documents related to the Message Broker integration:

- [draft-ietf-nmop-yang-message-broker-integration](#) defines the YANG-Push to message broker integration architecture.
- [draft-ietf-nmop-message-broker-telemetry-message](#) defines the message schema being used between YANG data producer and consumer.
- **This document:** [draft-ietf-nmop-yang-message-broker-message-key](#) defines how YANG-Push metrics can be indexed and addressed efficiently and therefore reduce the number of consumed messages and reduce the number of stored metrics with topic compaction.

At IETF 124 we introduced the keys for message broker concepts; at IETF 125 the successful working group adoption and the hackathon proof of concept. Today we present revision -02, addressing the hackathon findings, and the first open-source implementation of message keys and topic names.

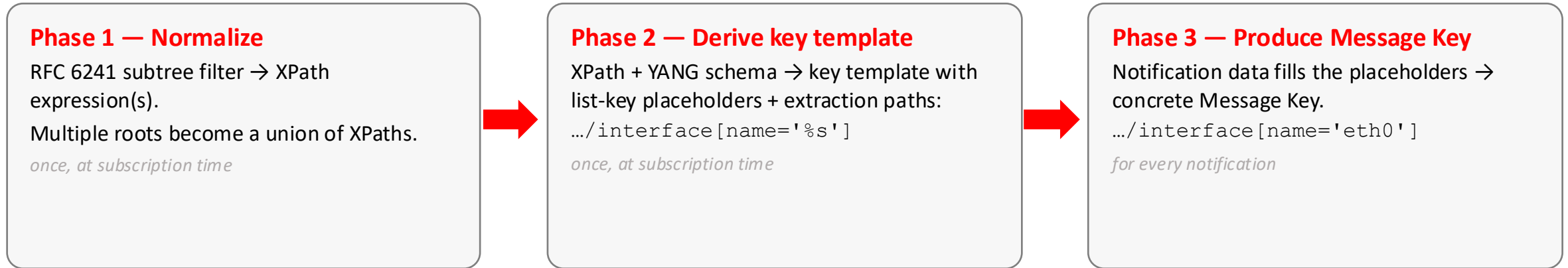
What's New in Revision -02

Major changes since the last presented revision

- **Message Key and Topic Name derivation fully specified** as a three-phase algorithm, addressing the IETF 125 hackathon findings (next slides).
- **Message Key format is defined**
- **Terminology refined:** a Subject now corresponds to a unique schema tree within a Schema Registry and identifies Messages within a Topic.

Three-Phase Message Key Algorithm

As specified in the draft



Message Key = three line-delimited fields: node name, YANG-Push subscription ID, and the sorted concrete XPath(s):

- router-01 • 1042 • /ietf-interfaces:interfaces/interface[name='eth0']

Deterministic by design: the same data instance always produces the same key — the basis for partition placement and topic compaction.

Phases 1 and 2 run once at subscription time; only Phase 3 runs per notification — minimal overhead at delivery.

Topic Name Derivation Algorithm

As specified in the draft — schema path to broker-safe name

Derived from the Phase 2 key template — a pure, schema-path-only transformation:

1. **Strip predicates** to get the pure schema path: `/ietf-interfaces:interfaces/interface`
2. **Substitute module names** with their YANG prefix statement: `ietf-interfaces` → `if`
3. **Flatten** — drop the leading '/'; ':' and '/' become '-': `if-interfaces-interface`
4. **Prepend an organization prefix** (optional): `netops-if-interfaces-interface`
5. **Truncate** names exceeding the broker limit (Kafka: 249 chars) at a '-' boundary and append a hash suffix.

Properties:

- **Deterministic** — a pure function of the schema path — and **collision-free** — the mapping is injective, no two schema paths share a topic name.
- **Stable under schema evolution** — augmenting a module adds new topic names but never changes existing ones.
- **Regex- and wildcard-friendly** — `if-interfaces-interface-.*` matches all interface metrics, `if-.*` everything from `ietf-interfaces` — and **human readable**: prefix identifies the module, node names stay unabbreviated.

IETF 125 Hackathon Learnings

Issues found in the proof of concept — addressed in -02

- **Topic naming restrictions:** topic derivation now uses YANG module prefixes and '-' separators only — collision-free mapping, length capped with a hash suffix, safe for Kafka (no ':', no '_' and '.' mix).
- **Subtree filter underspecified:** Phase 1 defines the normalization of subtree filters into XPath; multiple roots become a union of branches.
- **Message Key format not fully defined:** now explicit — node name, subscription ID and sorted concrete XPaths as line-delimited fields; separator and datastore ambiguities resolved.
- **Key derivation for lists underspecified:** key templates cover nested lists, composite (multi-key) lists and key value quoting and escaping.
- **XPath is more expressive than a simple path:** defined handling for union expressions, positional predicates and predicates on non-key leaves.
- **Module revisions, features and prefixes:** the schema context is loaded from the YANG Library (RFC 8525), including module revisions and features; topic names use the module's own prefix statement.

Thanks to the hackathon team: Thomas Graf, Wanting Du, Maxence Younsi and Pierre Francois.

First Implementation: yang-push-key

github.com/ahassany/yang-push-key

Implements the Message Key and Topic Name algorithms as specified in the draft. Open-source Rust library and CLI (Apache 2.0), YANG schema aware through libyang. See: github.com/ahassany/yang-push-key

- The CLI mirrors the draft's phases — `phase1`, `phase2`, `phase3`, `pipeline` and `topic` — so each step of the specification can be validated and debugged independently.
- Schema context from individual YANG modules or an RFC 8525 YANG Library, including features.
- **Example:** `topic /ietf-interfaces:interfaces/interface` $\Rightarrow$ `netops-if-interfaces-interface`
- Covers composite keys, nested lists, leaf-lists, positional predicates and quoting edge cases — validated by 116 tests.

Feedback and additional implementations welcome!

Status and Next Steps

Revision -02, First Implementation, Feedback

Status

- **Revision -02 posted.** It fully specifies the Message Key and Topic Name derivation, addressing the IETF 125 hackathon findings, and promotes the telemetry message and notification envelope drafts to normative references.
- **First open-source implementation available:** github.com/ahassany/yang-push-key validates the Message Key and Topic Name algorithms end to end against real YANG schemas and notifications.

Next Steps

- **NetGauze® integration.** Integrate the open-source implementation library in NetGauze YANG-Push collector and test the full chain.